

19-Functions in Python

Introduction to Functions

- Functions in Python are reusable pieces of code allowing for repeated use without redundancy.
- The importance of the DRY (Don't Repeat Yourself) principle is emphasized to avoid code duplication.

Function Definition and Syntax

- A function is defined using the `def` keyword, followed by the function name and parentheses (with optional parameters), then a colon.
- Example of function definition:

```
def my_function(param1, param2):  
    # code here  
    return result
```

- Function parameters are optional; a function may have none, one, or several.
- Indentation is essential: all function code must be indented beneath the `def` statement.
- The return statement is optional—if omitted, Python returns `None` by default.

Function Execution and Calling

- Defining a function creates the function object in memory but does not execute its code.
- To run the code inside a function, you must call it by name (with appropriate arguments).

- Example: `my_function(arg1, arg2)`
- When a function returns a value, that value can be captured or used directly (e.g., inside a print statement).

Return Statements

- The return statement provides output from a function back to the caller.
- If no return is specified, the function returns `None` by default.
- Every function in Python returns something, either an explicit value or `None`.
- The return value is considered the function's output.

Types of Parameters and Arguments

Positional Parameters/Arguments

- The order of arguments matters; values are assigned to parameters based on position.
- The number of arguments must match the number of parameters.
- Changing the order or mismatching number of arguments/parameters results in errors or unintended behavior.
- Example: Employee info function where name, id, salary, and location must be assigned correctly by order.

Keyworded and Default Arguments

- Keyword arguments allow assignment of values by the parameter name, making order irrelevant.
- Default arguments provide fallback values for parameters if no value is provided during the function call.
- Default parameters must come after positional parameters in the function definition.
- If a keyword argument is passed, it overrides the default value.
- Example: Filtering a list for 'male' values with a default gender parameter.

Variable-Length Arguments

- Used when the number of arguments is unknown or can vary.
- `*args` (e.g., `*numbers`) collects extra positional arguments as a tuple.
- `**kwargs` collects extra keyword arguments as a dictionary.
- Function examples: Summing any number of integers, form submissions with variable optional fields.
- Mandatory positional arguments must precede variable-length arguments.
- Demonstrated difference between variable-length positional and keyworded arguments.

Common Errors and Best Practices

- Placing default parameters before non-defaults causes syntax errors.

- Providing too few or too many positional arguments results in errors.
- Mixing positional and keyword arguments requires care: all positional must come before keyword arguments when calling a function.
- Type mismatches (e.g., trying to add a string and integer) raise runtime errors.
- Functions should be used to encapsulate code blocks for reusability and clarity.

Q&A and Closing Remarks

- Instructor answered questions regarding function argument handling and code behavior.
 - The class will progress to recursion, lambda, filter, map, and reduce functions in the next session.
 - Assignment solutions will be provided after all submissions are complete.
-

This summary presents all essential concepts, usage patterns, caveats, and illustrative examples covered in the lecture, maintaining the instructional tone and logical flow of the original document.